

The Bugs

- **Array Filter Bug:** Inside `app.get('/tasks')`, the filter method `filteredTasks.filter(...)` does not mutate the array in place. Because its returned result is never reassigned back to `filteredTasks`, the `?done=` query parameter is completely ignored.
- **Missing Seed Array Bracket:** The `initialTasks` seed data array is missing an opening array bracket `[` or an object is trailing outside the array. It is written as:

javascript

```
const initialTasks = [  
  { id: 1, title: "Buy groceries", done: false },  
  { id: 2, title: "Clean the room", done: true },  
]; // <-- Array mistakenly closed early here  
{ id: 3, title: "Study API design", done: false } // <-- Syntax error (isolated object
```

Use code with caution.

Fixed Node.js `server.js` Code

Here is the fully corrected code with the array assignment fixed and the `initialTasks` array syntax repaired:

javascript

```
const express = require('express');  
const app = express();  
const PORT = 3000;  
  
app.use(express.json());  
  
// FIXED: Corrected array nesting brackets  
const initialTasks = [  
  { id: 1, title: "Buy groceries", done: false },  
  { id: 2, title: "Clean the room", done: true },  
  { id: 3, title: "Study API design", done: false }  
];
```

```
// In-memory runtime database array
let tasks = [...initialTasks];

app.get('/', (req, res) => {
  res.json({ name: "Task API", version: "1.0", endpoints: ["/tasks", "/stats"] });
});

app.get('/health', (req, res) => {
  res.json({ status: "ok" });
});

// GET /tasks Supports ?done=true/false and ?search=word parameters
app.get('/tasks', (req, res) => {
  let filteredTasks = [...tasks];
  const { done, search } = req.query;

  // 1. Filter by completed status flag
  if (done !== undefined) {
    const isDone = done === 'true';
    // FIXED: Now properly reassigning the filtered array result
    filteredTasks = filteredTasks.filter(t => t.done === isDone);
  }

  // 2. Filter by matching title keyword text
  if (search !== undefined && search.trim() !== "") {
    filteredTasks = filteredTasks.filter(t =>
      t.title.toLowerCase().includes(search.toLowerCase())
    );
  }

  res.json(filteredTasks);
});

// GET /stats Calculates summary information
app.get('/stats', (req, res) => {
  const total = tasks.length;
  const done = tasks.filter(t => t.done).length;
  const open = total - done;
  res.json({ total, done, open });
});

// POST /reset Re-seeds initial values
app.post('/reset', (req, res) => {
```

```
tasks = [...initialTasks];
res.json({ message: "Database reset to initial seed values", count: tasks.length });
});

// Standard CRUD endpoints below
app.get('/tasks/:id', (req, res) => {
  const taskId = parseInt(req.params.id);
  const task = tasks.find(t => t.id === taskId);
  if (!task) return res.status(404).json({ error: `Task ${taskId} not found` });
  res.json(task);
});

app.post('/tasks', (req, res) => {
  const { title } = req.body;
  if (!title || title.trim() === "") return res.status(400).json({ error: "Title is required" });
  const nextId = tasks.length > 0 ? Math.max(...tasks.map(t => t.id)) + 1 : 1;
  const newTask = { id: nextId, title, done: false };
  tasks.push(newTask);
  res.status(201).json(newTask);
});

app.put('/tasks/:id', (req, res) => {
  const taskId = parseInt(req.params.id);
  const taskIndex = tasks.findIndex(t => t.id === taskId);
  if (taskIndex === -1) return res.status(404).json({ error: `Task ${taskId} not found` });

  const { title, done } = req.body;
  if (title === undefined && done === undefined) return res.status(400).json({ error: "Missing fields" });
  if (title !== undefined && (!title || title.trim() === "")) return res.status(400).json({ error: "Title is required" });
  if (done !== undefined && typeof done !== "boolean") return res.status(400).json({ error: "Done must be a boolean" });

  if (title !== undefined) tasks[taskIndex].title = title;
  if (done !== undefined) tasks[taskIndex].done = done;
  res.json(tasks[taskIndex]);
});

app.delete('/tasks/:id', (req, res) => {
  const taskId = parseInt(req.params.id);
  const taskIndex = tasks.findIndex(t => t.id === taskId);
  if (taskIndex === -1) return res.status(404).json({ error: `Task ${taskId} not found` });
  tasks.splice(taskIndex, 1);
  res.status(204).send();
});
```

```
app.listen(PORT, () => console.log(`Server on http://localhost:${PORT}`));
```